Running the proofs: the DOA learning simulation

Siegler & Shrager's 1984 simulation, rerun from CogLean's own definitions

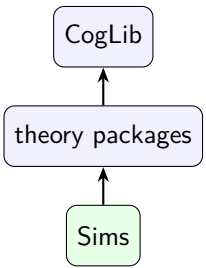
Jeff Shrager

October 2026

Executable and provable: the same definitions

- A CogLean model is a computable Lean definition: the theorems are about it, and it runs.
- Until now, runs were **exact**: every outcome with its exact probability. That is how the worked cases are proved.
- Exact runs do not scale: a 2000-trial learning session branches into astronomically many histories.
- So: **sampling**, one outcome at a time, from the *same* definitions.
- No second implementation to drift: earlier experiments used Python mirrors of the Lean; these runs use the Lean itself.

How it is organized



- A separate package; arrows point one way: nothing in the library or theories imports it.
- A **seeded** generator: every run reproducible.
- **Exact** draws: a uniform $k/10^9$ against the exact cumulative probabilities; no floating point in sampling.
- Programs: `lake exe sc-check`, `lake exe sc-learning`.

First, check the sampler against the model

- A trial is drawn stage by stage, using the model's own retrieval, elaboration and learning; counting object by object.
- On small cases, compare every outcome's frequency with its *exact* probability under the model's `trial`:

Case (40,000 draws each)	worst $\lvert z \rvert$	within 4σ
$1+1$, search length 1, criterion .3	1.30	yes
$1+2$, search length 2, criterion .2	1.60	yes
$2+2$, search length 3, criterion .5	1.25	yes

So the simulation draws from the model, not from an approximation of it.

The simulation in the 1984 chapter (pp. 272–275)

Specified:

- answers 1–12, every association initially .01;
- 2000 learning trials; the 25 problems presented by parental input (Table 9.5), plus a constant;
- on each trial, a criterion from .05–.99 and a search length from 1–3, at random;
- elaboration adds .05; a counting-string prime on ascending and tie problems;
- learning: twice the increment for correct answers.

Not given, so parameters here: the prime's size, the increment, the counting error rates, the presentation constant ($c = 12$ reproduces the stated 5.4% for $4 + 1$).

Searching the unspecified parameters

Mean |difference| from Fig. 9.8's peaks and shares (10 runs each):

increment \ prime	.01	.02	.05	.1	.2
1/400	.18	.18	.18	.18	.18
1/250	.11	.10	.10	.09	.08
1/200	.10	.09	.09	.08	**.06**
1/150	.18	.17	.18	.17	.14
1/100	.43	.42	.44	.41	.40

An increment near 1/200 fits; faster learning overshoots. The fit improves with a *larger* prime.

Against Fig. 9.8, after 2000 trials

Problems (effect)	Peak	Chapter	Share	Chapter
$1+4$ vs $2+3$ (counting string)	.54 vs .40	.67 vs .37	.75 vs .61	.77 vs .46
$4+1$ vs $3+2$ (frequency)	.85 vs .55	.80 vs .51	.78 vs .71	.76 vs .69
$3+4$ vs $4+5$ (sum)	.31 vs .32	.39 vs .32	.55 vs .52	.50 vs .43

Increment $1/200$, prime .2, counting errors .05; means over 20 runs. Peak: strength at the correct answer. Share: its fraction of the total.

What reproduces, and what does not

- **Frequency**: reproduced closely, at every setting tried.
- **Counting string**: reproduced in direction, but only with a large prime, .2, four times the elaboration's .05. The chapter does not give its size; this suggests it was large.
- **Sum**: not reproduced at any setting tried (primes to .4; counting errors .02–.1). Possible reasons:
  - Table 9.5, as read from the scan, gives $4 + 5$ slightly more input than $3 + 4$; the text says equal. Some cells are faint.
  - The chapter's values are from one simulated run; these are means over 20.
  - Unspecified details: a continuous criterion, the prime during elaboration, the form of counting errors.

What this adds, and next

- **One implementation**, proved about and run; a sampler checked against the exact model before any long run.
- **Replications become specific**: where a published simulation leaves numbers out, the sweep shows which values matter, and what they must have been.
- **Disagreements become questions**: the sum effect points at a table, a single run, or an unstated detail.

Next: run-to-run spread; recheck Table 9.5; the test phase (Fig. 9.2A); then the multi-digit model learning as a whole.